Sovereign AI

Technical Roadmap: Building a Domain-Specific BFSI LLM

12-Phase Pipeline | 32,000-token BFSI Vocabulary | Hugging Face + Ollama

---

Data + Trust + Intelligence for a Sovereign Future

Prepared for: Sovereign AI project
Date: 22 September 2026

Framing Decision: Track A vs Track B

"Sovereign AI" plus a 32,000-token BFSI vocabulary suggests genuine data/model sovereignty may matter (not just fine-tuning a foreign checkpoint). Both paths below are legitimate — choose based on your real constraint: compute budget vs. sovereignty requirement.

Dimension	Track A — Domain-adapt an open-weight base (recommended)	Track B — Pretrain from scratch (full sovereignty)
Base	Start from Llama 3.1/3.2, Mistral 7B, Qwen2.5, or Gemma 2 (open weights, inspectable, already knows grammar/reasoning)	Random-init transformer, trained entirely on your corpus
Tokenizer	Extend existing tokenizer with ~32K BFSI merges, or train a fresh 32K SentencePiece tokenizer for continued pretraining	Fresh 32K BPE/SentencePiece tokenizer — 32K matches Llama-2's proven vocab size
Compute	1–8 GPUs (A100/H100, or RTX 4090-class with QLoRA), days to 2 weeks	32–100+ GPUs for weeks; 50–300B tokens minimum for a usable small model
Data need	1–10B domain tokens is workable	100B+ tokens or the model collapses into gibberish
Realistic for	Most teams	Only with a dedicated ML infra team + budget, or a hard regulatory mandate against foreign-trained weights

The phase-by-phase pipeline below defaults to Track A (fastest path to a working BFSI assistant); Track B divergences are called out inline.

1 Data Collection

What it involves:

Assembling the raw BFSI corpus — regulatory filings, product docs, policy wordings, KYC/AML guidelines, RBI/SEBI/IRDAI circulars, annual reports, call-center transcripts, internal SOPs, FAQs, contracts.

Steps:

- Inventory sources: internal document stores, public regulator sites (RBI, SEBI, IRDAI, FATF), Hugging Face finance datasets, earnings call transcripts.
- Build ingestion connectors per source type: requests/scrapy for web, IMAP/API pulls for email, DB exports, OCR for scanned PDFs (Tesseract, PaddleOCR, or cloud OCR at scale).
- Tag every document at ingestion with source, date, sensitivity level, and license — this drives dedup, PII filtering, and later data-governance audits.
- Store raw data in an object store (S3/Azure Blob/MinIO) with a manifest (Parquet/JSON) — never mutate raw files in place.

Tools:

requests, scrapy, BeautifulSoup, PyPDF2/pdfplumber, Tesseract/PaddleOCR, Whisper (call-center audio), Airflow or Dagster for orchestration.

BFSI note:

Much of this is regulated data (customer PII, KYC docs). Decide now what legally cannot leave your environment — this determines whether Track B is actually required.

2 Data Cleaning & Chunking

What it involves:

Turning messy raw text into clean, deduplicated, PII-scrubbed chunks ready for tokenization.

Steps:

- Normalize encoding (force UTF-8), strip HTML/boilerplate, fix OCR artifacts.
- Deduplicate — exact hash dedup first (MD5/SHA), then near-dedup with MinHash/LSH (datasketch) to catch reformatted copies of the same circular.
- PII/sensitive-data removal — mask account numbers, PAN/Aadhaar, card numbers, names using regex + a NER model (spaCy, Microsoft Presidio).
- Chunk documents into training-sized windows (512–2048 tokens) with ~10–15% overlap so context isn't severed mid-clause.
- Quality filter: drop chunks that are mostly garbled tables, boilerplate headers/footers, or below a language-quality threshold.

Tools:

datasketch (MinHash), Microsoft Presidio, spaCy, ftfy, langdetect/fastText, the unstructured library.

Output:

A clean JSONL corpus: {"text": ..., "source": ..., "doc_id": ..., "chunk_id": ...}.

3 Tokenizer & Vocabulary Building — the 32K BFSI Vocab

What it involves:

Building the subword vocabulary the model will actually see as input IDs.

Track A steps:

- Take the base model's tokenizer (e.g. Llama 3's 128K-token tokenizer).
- Run BPE/Unigram training only on your BFSI corpus with the Hugging Face tokenizers library to discover domain-specific merges ("NPA", "CASA ratio", "IRDAI", "moratorium", ISIN codes, SWIFT/IFSC patterns).
- Add the highest-value ~2,000–5,000 new tokens via tokenizer.add_tokens(), resize the model's embedding matrix, and initialize new rows from the mean of existing embeddings.

Track B steps:

- Train a SentencePiece BPE model directly on the full BFSI corpus with vocab_size=32000 (matches Llama-2's proven budget).
- Reserve special tokens: , , , , plus optional BFSI structural tokens (, ,).
- Validate coverage: run the tokenizer over a held-out BFSI sample; OOV/byte-fallback rate should be <1–2%, and key domain terms should tokenize as 1–2 tokens, not fragment into 5+.

Tools:

Hugging Face tokenizers (Rust-backed), sentencepiece, tiktoken (if targeting GPT-style BPE).

4 Embedding Generation

What it involves:

Converting token IDs into dense vectors. Two distinct meanings apply — don't conflate them: the model's own trainable input-embedding table, and a separate retrieval-embedding model used for RAG at inference time.

Steps:

- Model's own embeddings: Track A copies from the base model and mean-inits new rows; Track B random-inits (e.g. Xavier/normal(0, 0.02)).
- Retrieval/RAG: pick or fine-tune a sentence-embedding model — BAAI/bge-large-en, intfloat/e5-large-v2, or fine-tune one on BFSI query-passage pairs for better domain recall.
- Store retrieval embeddings in a vector DB: FAISS (local/simple), Qdrant, Weaviate, or pgvector if you already run Postgres.

Tools:

Hugging Face sentence-transformers, FAISS, Qdrant.

5 Dataset Preparation

What it involves:

Splitting and packaging the tokenized corpus for training.

Steps:

- Split train/val/test — typically 95/2.5/2.5 for pretraining corpora; stratify by source so val/test aren't dominated by one document type.
- Tokenize the full corpus with the Phase 3 tokenizer; pack sequences to a fixed context length (e.g. 4096) using concatenation + document-boundary tokens.
- Shard into multiple files (.arrow/.parquet/WebDataset tar shards) for streaming during multi-GPU training — don't load everything into RAM.
- For fine-tuning, build a separate instruction dataset: BFSI Q&A pairs and task instructions ("summarize this credit memo", "flag AML red flags in this transaction narrative"), formatted as chat turns.

Tools:

Hugging Face datasets (streaming mode), WebDataset, Parquet/Arrow.

6 Model Pretraining

Track A (recommended):

Skip full pretraining — instead run continued/domain-adaptive pretraining (DAPT): keep training the open-weight base (causal LM objective) on your cleaned BFSI corpus for a few epochs at low learning rate. Teaches BFSI vocabulary/style/facts without needing 100B+ tokens.

Track B (from scratch):

Full causal language model pretraining — next-token prediction over your entire corpus from random init.

Steps:

- Choose architecture — fixed by the base model in Track A; for Track B, a decoder-only transformer (Llama-style: RMSNorm, SwiGLU, RoPE) at 1–3B params is realistic for a single-org build.
- Set up distributed training: torch.distributed / DeepSpeed ZeRO / FSDP for multi-GPU sharding; use mixed precision (bf16).
- Optimizer: AdamW, cosine LR schedule with warmup, weight decay ~0.1.
- Track loss/perplexity continuously; checkpoint every N steps to resume from failures (this stage runs days–weeks).
- Log loss curves, gradient norms, LR to Weights & Biases or MLflow.

Tools:

Hugging Face transformers + accelerate, DeepSpeed, PyTorch FSDP, nanoGPT/litgpt (from-scratch references), Weights & Biases.

7 Model Fine-tuning

What it involves:

Turning the base/continually-pretrained model into a BFSI assistant that follows instructions.

Steps:

- SFT (Supervised Fine-Tuning) on the instruction dataset — format as chat turns, mask loss on user/system tokens so the model only learns to predict assistant responses.
- Use QLoRA (4-bit quantized base + LoRA adapters) if GPU budget is limited — trains a 7–8B model on a single 24GB GPU. Use full-parameter SFT only with multi-GPU headroom.
- RAG integration (alongside fine-tuning, not instead of it): connect the Phase 4 vector store so the model retrieves current regulations/policies rather than relying on memorized, potentially stale facts.
- Optional second stage: DPO (Direct Preference Optimization) using preference pairs to sharpen tone, compliance-safety, and refusal behavior — simpler than full RLHF and increasingly the standard choice.

Tools:

Hugging Face peft (LoRA/QLoRA), trl (SFTTrainer, DPOTrainer), bitsandbytes (4-bit quantization), axolotl (simplifies SFT/DPO config).

8 Evaluation & Benchmarking

What it involves:

Measuring whether the model actually got better and is safe to ship.

Steps:

- Standard NLP metrics on held-out test set: perplexity, and BLEU/ROUGE/F1 where reference answers exist.
- Build a domain benchmark suite yourself — ~200–500 curated BFSI Q&A/task pairs with graded answers (regulatory interpretation, financial calculation correctness, document summarization accuracy).

- Use an LLM-as-judge (a stronger model scoring outputs against a rubric) to scale qualitative evaluation — spot-check the judge against human ratings.
- Run general-capability regression checks (MMLU subset, TruthfulQA) to confirm fine-tuning didn't degrade base reasoning.

Tools:

Hugging Face evaluate and lighteval, lm-evaluation-harness (EleutherAI), custom BFSI eval harness.

9 Alignment & Safety

What it involves:

Ensuring the model won't give harmful, non-compliant, or fabricated financial/regulatory advice.

Steps:

- Build a BFSI-specific red-team set: prompts probing incorrect regulatory advice, PII leakage, biased credit decisions, hallucinated numbers presented as fact.
- Apply DPO/RLHF weighted toward refusing unsafe requests and citing sources for factual/regulatory claims.
- Add a system-prompt/guardrail layer: mandatory disclaimers on financial advice, refusal templates for account-specific PII requests without verified auth context.
- Bias testing across protected attributes in any credit/underwriting-adjacent outputs — a real regulatory exposure area (model risk frameworks such as SR 11-7 and RBI's AI-in-lending guidance apply).

Tools:

trl (DPO), NeMo Guardrails or Guardrails AI, custom red-team prompt sets, Presidio as an output filter.

10 Model Optimization

What it involves:

Shrinking the model for practical inference latency/cost without losing too much quality.

Steps:

- Quantization: convert to 4-bit/8-bit — GGUF via llama.cpp (the format Ollama consumes), or AWQ/GPTQ for GPU serving.
- Pruning/distillation (optional): distill the fine-tuned BFSI model into a smaller student model for edge/low-latency deployment.
- KV-cache optimization: enable paged attention (vLLM does this automatically) for efficient multi-request serving.
- Benchmark latency/throughput/VRAM at each quantization level before picking a production config — Q4_K_M is usually the GGUF sweet spot.

Tools:

llama.cpp (GGUF conversion, the Ollama path), AutoAWQ/AutoGPTQ (vLLM/TGI serving), Hugging Face optimum.

11 Deployment & Serving

What it involves:

Two realistic serving paths, chosen by inference volume: Ollama for internal tools/dev/moderate single-node load; vLLM/TGI for production scale.

Steps:

- Ollama path: convert the fine-tuned model to GGUF, write a Modelfile (system prompt, template, params), run 'ollama create bfsi-model -f Modelfile', serve via Ollama's REST API or OpenAI-compatible endpoint.
- Hugging Face stack path: serve via vLLM or Text Generation Inference (TGI) behind a FastAPI gateway, containerized, on Kubernetes with GPU node pools, autoscaled on request-queue depth.
- Package the model artifact (GGUF for Ollama, or safetensors + tokenizer config for vLLM/TGI).
- Stand up an API layer (FastAPI) handling auth, rate limiting, prompt templating, and RAG retrieval calls before hitting the model.
- Containerize (Docker), deploy via Kubernetes or Docker Compose, with GPU scheduling.
- Put it behind a gateway that logs every request/response — needed for Phase 12 and for BFSI audit trails.

Tools:

Ollama, vLLM, TGI, FastAPI, Docker, Kubernetes, NGINX/Traefik.

12 Monitoring & Continuous Improvement

What it involves:

Keeping the model correct and compliant after launch — BFSI regulations change, so a static model goes stale.

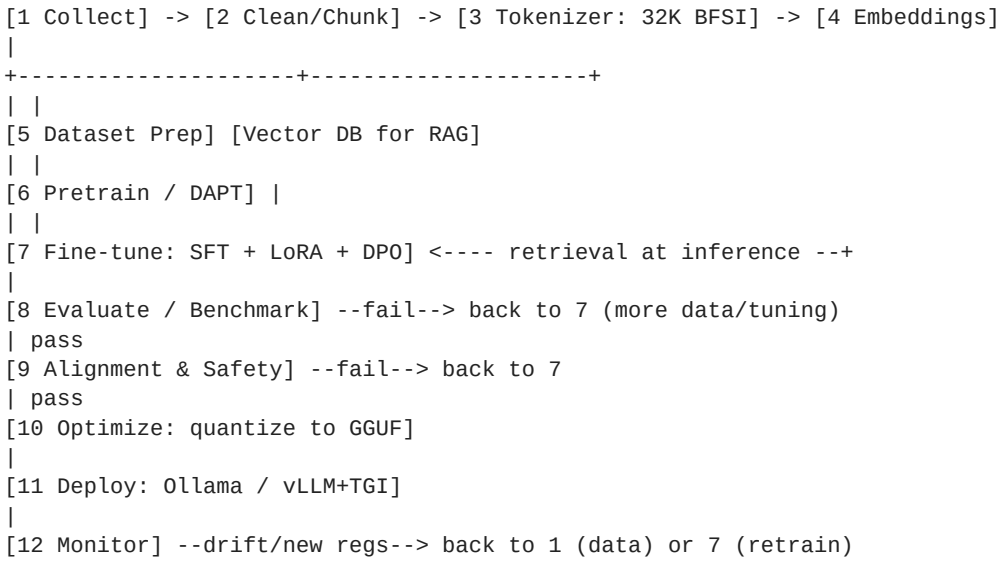
Steps:

- Log all inputs/outputs (with PII redaction applied to logs); track latency, error rate, and refusal rate on a dashboard.
- Collect user feedback (thumbs up/down, corrections) and route flagged responses into a review queue.
- Drift detection: periodically re-run the Phase 8 benchmark suite against production traffic samples to catch quality degradation.
- Set a retraining cadence: refresh the RAG knowledge base continuously (cheap, no retraining needed), but schedule periodic DAPT/SFT refreshes (e.g. quarterly) to absorb new regulations and corrected feedback — this closes the loop back to Phase 1.

Tools:

Prometheus + Grafana, MLflow (versioning across retraining cycles), a feedback-capture UI, Airflow/Dagster to schedule the retraining pipeline.

How the 12 Phases Connect Into One Pipeline



Orchestrate this as a DAG (Airflow or Dagster) so Phases 1–5 (data pipeline) can run independently and re-trigger Phase 7 (retrain) on a schedule without re-running the whole thing from scratch.

Consolidated Tech Stack

Layer	Tools
Ingestion / cleaning	requests, unstructured, Presidio, datasketch, ftfy
Tokenizer	Hugging Face tokenizers, sentencepiece
Data / training format	Hugging Face datasets, Parquet/Arrow, WebDataset
Pretraining	transformers, accelerate, DeepSpeed/FSDP, litgpt (Track B)
Fine-tuning	peft, trl, bitsandbytes, axolotl
Retrieval	sentence-transformers, FAISS/Qdrant
Evaluation	lm-evaluation-harness, lighteval, custom BFSI eval set
Safety	NeMo Guardrails / Guardrails AI, Presidio
Optimization	llama.cpp (GGUF), AutoAWQ/AutoGPTQ
Serving	Ollama, vLLM, TGI, FastAPI
Infra	Docker, Kubernetes, Airflow/Dagster
Observability	MLflow, Weights & Biases, Prometheus, Grafana

Recommended First Move

With an empty project directory today, start with Track A end-to-end on a small slice (Phases 1–3 and 7 only, skipping full pretraining) to get a working prototype in days rather than months: pick one open base model (Llama 3.1 8B or Mistral 7B via Hugging Face), build a few-hundred-document BFSI corpus, extend its tokenizer, LoRA fine-tune, and serve via Ollama. That validates the whole pipeline shape before committing to the heavier Phase 6 pretraining decision.